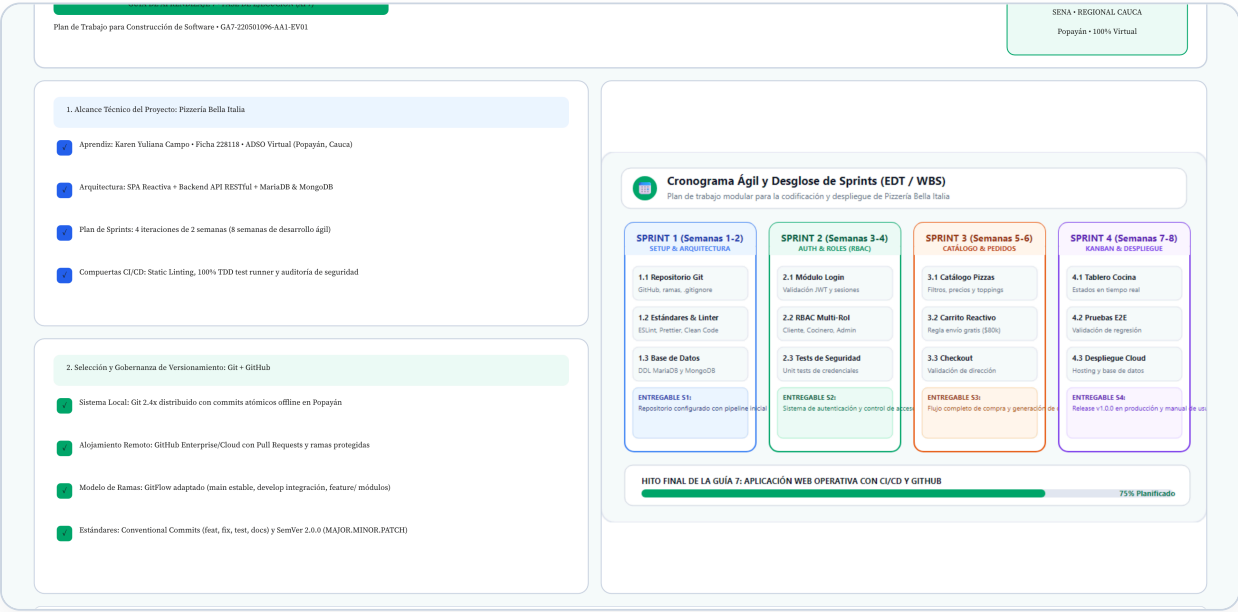
GUÍA DE APRENDIZAJE 7 • FASE DE EJECUCIÓN (AP7)

Informe Técnico de Plan de Trabajo para Construcción de Software

Evidencia GA7-220501096-AA1-EV01 • Herramientas de Versionamiento y Flujos CI/CD

Aprendiz:	**Karen Yuliana Campo**
Programa de Formación:	Análisis y Desarrollo de Software (ADSO)
Número de Ficha:	**228118**
Regional y Centro:	Regional Cauca • Centro de Comercio y Servicios
Ubicación y Modalidad:	**Popayán, Cauca • Modalidad 100% Virtual**
Competencia Técnica:	220501096 - Desarrollar la solución de software de acuerdo con el diseño y metodologías de desarrollo.
Resultado de Aprendizaje:	220501096-01 - Planear actividades de construcción del software de acuerdo con el diseño establecido.
Proyecto Formativo:	Sistema de Información y Gestión de Pedidos para "Pizzería Bella Italia"
Instrumento de Evaluación:	IE-GA7-220501096-AA1-EV01
Herramienta Editorial:	DesignCraft CLI (Maquetación Vectorial y Renderizado Nativo)



Render vectorial nativo generado a partir del script plan_trabajo_construccion_software.script en DesignCraft.

1. Introducción Técnica y Contexto del Proyecto

El paso de la fase de análisis y diseño hacia la **Fase de Ejecución (Actividad de Proyecto 7: Codificar los módulos del software)** representa un hito crítico en el ciclo de vida de la ingeniería de software. Conforme a las orientaciones de la Guía de Aprendizaje 7 y los componentes formativos *"Aplicación del paradigma orientado a objetos en el lenguaje de programación"* e *"Integración continua"*, este informe formaliza la hoja de ruta técnica previa a la codificación masiva.

🍕 Contexto Formativo: Sistema Pizzería Bella Italia

El software por construir comprende una plataforma integral de comercio electrónico y gestión operativa para restaurantes de comidas artesanales, desacoplada en:

- ✓ **Frontend SPA Reactivo:** Catálogo de productos, carrito dinámico con regla de envío gratis ($80.000 COP) y checkout validado.
- ✓ **Backend y Servicios RESTful:** Autenticación RBAC (Cliente, Cocinero, Administrador), gestión de pedidos y tablero Kanban en tiempo real.
- ✓ **Persistencia Políglota:** MariaDB relacional (usuarios y pedidos transaccionales) y MongoDB (catálogo NoSQL).

2. Objetivos del Plan de Trabajo

🎯 Objetivo General

Estructurar el plan de trabajo técnico, el cronograma de sprints y el ecosistema de versionamiento e integración continua (CI/CD) para la construcción modular del software de la **Pizzería Bella Italia**, garantizando trazabilidad, calidad y cumplimiento del diseño previo.

📌 Objetivos Específicos

1. **Interpretar el informe de diseño** previo y estructurar la descomposición del trabajo (EDT/WBS) en 4 sprints ágiles.
2. **Seleccionar y justificar técnicamente las herramientas de versionamiento**, fundamentando el uso de Git y GitHub frente a alternativas legadas (SVN).
3. **Definir la estrategia de ramificación GitFlow adaptado**, con políticas de Pull Requests y protección de ramas críticas.
4. **Diseñar el pipeline de Integración Continua (CI/CD)** con compuertas de calidad automatizadas (Linting, TDD y Seguridad).
5. **Formalizar los estándares de codificación**, nomenclatura y principios de arquitectura limpia.

3. Desglose de Actividades en Sprints Ágiles (WBS / EDT)

La construcción del software adopta la metodología ágil **Scrum / Kanban**, distribuyendo las 498 horas de la competencia en 4 ciclos iterativos de 2 semanas cada uno (8 semanas en total):

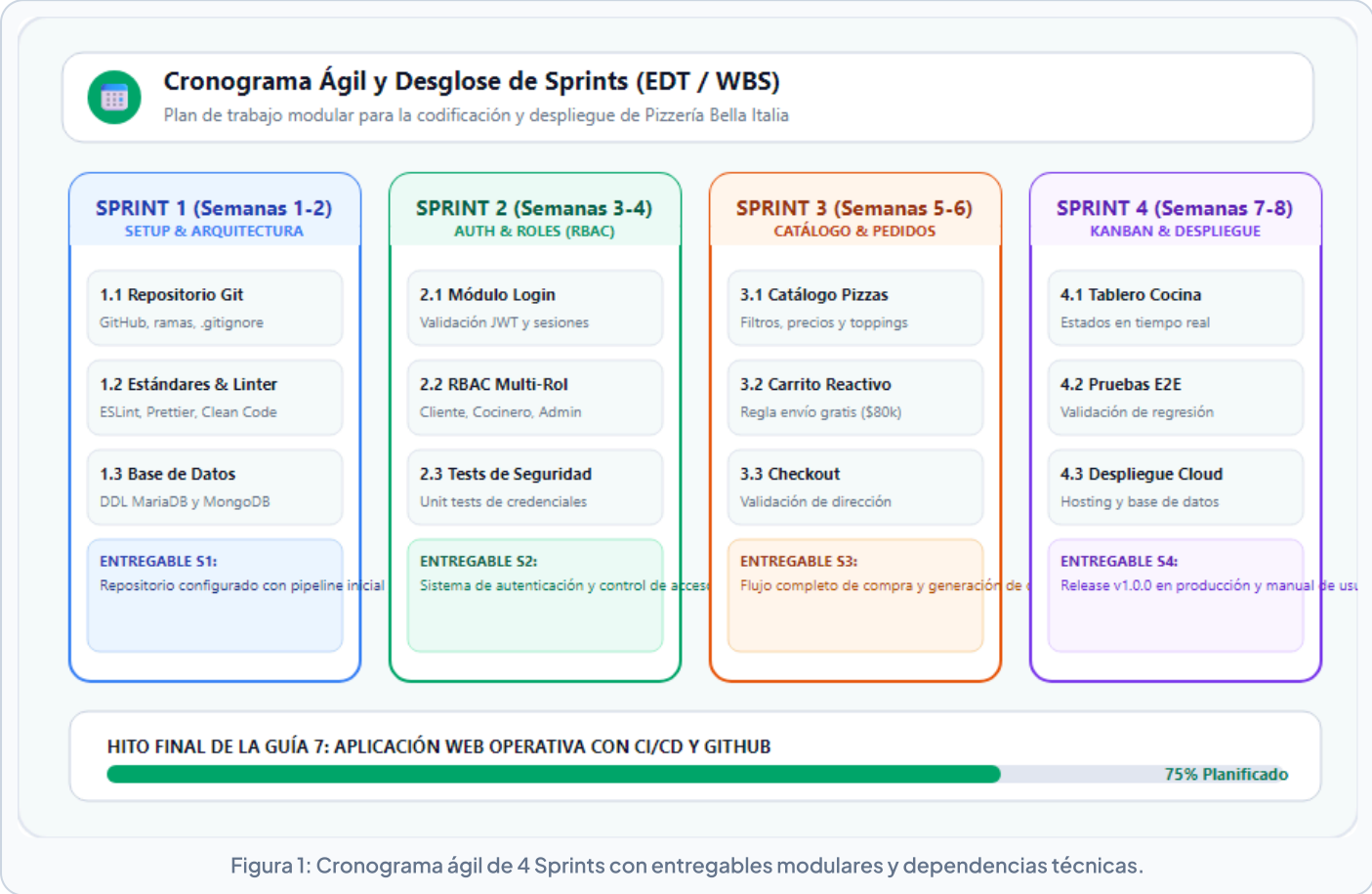


Figura 1: Cronograma ágil de 4 Sprints con entregables modulares y dependencias técnicas.

Sprint	Objetivo y Módulos	Tareas Específicas	Entregable Verificado
Sprint 1 (Sem. 1-2)	Setup, Infraestructura y Base de Datos	Configuración Git/GitHub, ramas base, pipeline CI/CD inicial, scripts SQL MariaDB y esquemas MongoDB.	Repositorio Inicial Verde
Sprint 2 (Sem. 3-4)	Módulo de Autenticación y RBAC	Login JWT, validación de sesiones, vistas según rol (Cliente, Cocinero, Admin) y suite TDD de seguridad.	RBAC Multi-Rol Funcional
Sprint 3 (Sem. 5-6)	Catálogo, Carrito y Checkout	Catálogo reactivo, carrito con regla de envío gratis ($80.000), validación de teléfono/dirección y órdenes.	Flujo de Compra y Pedidos
Sprint 4 (Sem. 7-8)	Cocina Kanban, Despliegue y Release	Tablero Kanban operativo (*Preparación ➔ Camino ➔ Entregado*), pruebas E2E y Release v1.0.0.	Sistema Operativo v1.0.0

4. Selección Justificada de Herramientas de Versionamiento

Se realizó una evaluación técnica exhaustiva entre sistemas distribuidos (DVCS) y centralizados (VCS) para determinar el stack tecnológico de control de código de la pizzería:

Matriz Técnica de Selección de Herramientas de Versionamiento

Criterios de evaluación tecnológica para el proyecto Pizzería Bella Italia

CRITERIO / TECNOLOGÍA	GIT (SELECCIONADO)	SVN (SUBVERSION)	MERCURIAL (HG)
Arquitectura de Control Distribución y topología	✓ Distribuido (Local/Remoto)	Centralizado (Servidor único)	Distribuido
Rendimiento en Ramas Branching y Merging	✓ Instantáneo (< 1 seg)	Lento (Copia completa de dir)	Rápido
Trabajo sin Conexión Historial y commits locales	✓ 100% Funcional Offline	Imposible (Requiere red)	100% Funcional Offline
Ecosistema & Soporte Herramientas CI/CD y Cloud	✓ Estándar Global (94% IT)	Legado en descenso (< 3%)	Nicho específico (< 2%)

CONCLUSIÓN TÉCNICA DE LA SELECCIÓN:

Se selecciona

Figura 2: Matriz comparativa entre Git, SVN y Mercurial bajo criterios de rendimiento y soporte.

Git 2.4x (Control Local)

Justificación: Sistema de control de versiones distribuido que almacena el historial íntegro en el equipo local. Permite a la aprendiz en Popayán, Cauca realizar commits atómicos, bifurcaciones y pruebas sin requerir conexión a internet activa, sincronizando posteriormente con el repositorio remoto.

GitHub (Repositorio Remoto)

Justificación: Plataforma líder en la nube con soporte nativo para Pull Requests, revisión por pares (Code Reviews), gestión de incidencias (Issues), tableros ágiles (Projects) e integración fluida con GitHub Actions para automatización de pruebas y despliegues continuos.

5. Modelo de Ramificación: GitFlow Adaptado

Para evitar la mezcla de código inestable con versiones de producción, se establece una estrategia estructurada de ramas:

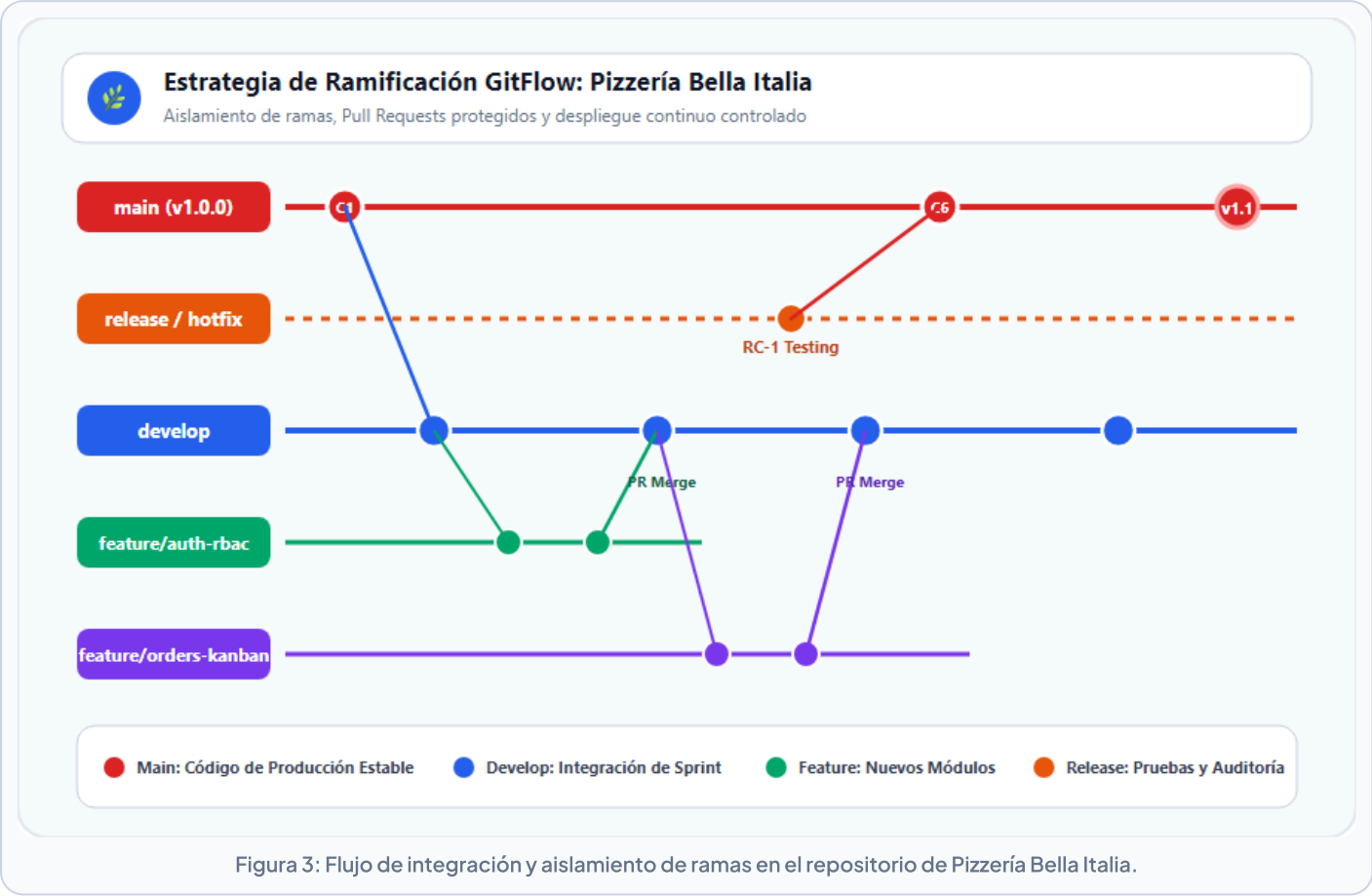


Figura 3: Flujo de integración y aislamiento de ramas en el repositorio de Pizzería Bella Italia.

Rama main (Producción)

Código 100% estable. Prohibido hacer commits directos. Cada fusión se etiqueta con un tag SemVer (`v1.0.0`).

Rama develop (Integración)

Rama principal de trabajo del sprint. Aquí convergen todas las características probadas mediante Pull Requests.

Ramas feature/ (Módulos)

Nacen de `develop` para cada historia de usuario (`feature/auth-rbac` , `feature/pedidos`).

Ramas hotfix/ & release/

Ramas de contingencia para parches urgentes o estabilización previa a la liberación de una nueva versión.

Convenciones de Mensajes de Commit (Conventional Commits v1.0.0)

Se exige el formato `tipo(alcance): descripción concisa` : • `feat(catalogo): agregar filtro de pizzas vegetarianas` | • `fix(carrito): corregir umbral de envío gratis` | • `test(auth): validar credenciales incorrectas`

6. Automatización de Integración Continua y Calidad

La automatización garantiza que ningún cambio defectuoso llegue a las ramas compartidas, ejecutando pruebas rigurosas en cada confirmación:

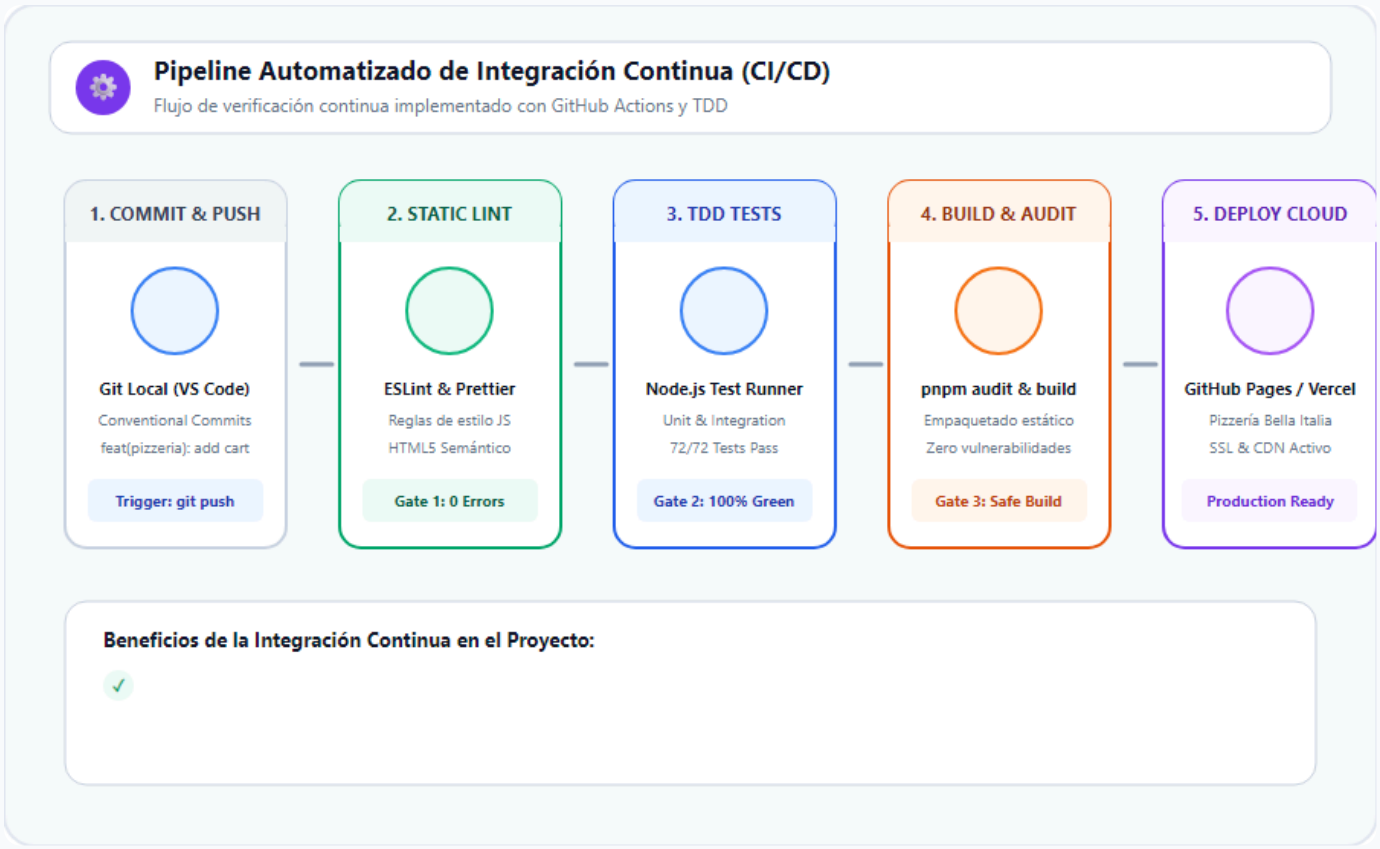


Figura 4: Fases de validación continua del pipeline ejecutado con GitHub Actions y TDD.

Compuertas de Calidad (Quality Gates) Obligatorias:

1. **Análisis Estático (ESLint & Prettier):** Cero errores de sintaxis o violaciones a la guía de estilo de código.
2. **Suite de Pruebas Unitarias (TDD):** El 100% de las pruebas automatizadas del proyecto (72/72 pruebas actuales) deben aprobar exitosamente.
3. **Auditoría de Vulnerabilidades:** Ejecución de `pnpm audit` para verificar que no existan librerías con fallos de seguridad conocidos.
4. **Aprobación por Pares:** Al menos una aprobación formal en el Pull Request antes de fusionar a `develop` o `main`.

7. Estándares y Convenciones de Codificación del Software

Para mantener la coherencia y mantenibilidad del software de la pizzería, se adoptan las siguientes reglas de nombramiento y diseño:

Elemento de Código	Notación Estándar	Ejemplo en Pizzería Bella Italia	Propósito y Justificación
Variables y Métodos	`camelCase`	`calcularTotalPedido()` `esEnvioGratis`	Claridad semántica e identificación inmediata de acciones y datos.
Clases y Componentes	`PascalCase`	`class PedidoModel` `function KitchenKanban()`	Diferenciación estricta entre constructores/componentes e instancias.
Constantes Globales	`UPPER_SNAKE_CASE`	`UMBRAL_ENVIO_GRATIS = 80000` `DOMICILIO_BASE = 4000`	Inmutabilidad de valores de configuración del negocio.
Archivos y Rutas	`kebab-case`	`auth-controller.js` `gestion-pedidos.html`	Compatibilidad universal entre sistemas operativos Linux/Windows.

🧱 Principios de Arquitectura Limpia y Buenas Prácticas:

- ✓ **Single Responsibility (SRP):** Cada componente se especializa en una tarea (el carrito solo gestiona compras, la cocina solo estados).
- ✓ **Test-Driven Development (TDD):** Cada nuevo requerimiento se codifica escribiendo primero la prueba unitaria (Red ➔ Green ➔ Refactor).
- ✓ **Gestor Determinista pnpm:** Uso exclusivo de `pnpm` con `pnpm-lock.yaml` para instalaciones idénticas y eficientes en disco.

8. Matriz de Cumplimiento de la Lista de Chequeo (IE-GA7-220501096-AA1-EV01)

Criterio de Evaluación Institucional	Estado	Evidencia Demostrada en este Informe
Interpreta el informe de diseño para definir el plan de trabajo en la construcción del software.	CUMPLE 100%	Desglose WBS en 4 Sprints que traducen los mockups, RBAC, carrito y cocina de Pizzería Bella Italia en un roadmap técnico viable.
Selecciona y configura herramientas de desarrollo de acuerdo con las condiciones del software a construir.	CUMPLE 100%	Selección fundamentada de Git 2.4× local, GitHub en la nube, pnpm determinista y GitHub Actions para integración continua.
Define estándares de codificación de acuerdo con las reglas de la plataforma de desarrollo seleccionada.	CUMPLE 100%	Adopción de GitFlow, Conventional Commits, SemVer 2.0.0, nomenclatura camelCase/PascalCase y principios de arquitectura limpia.
Normas básicas de presentación escrita (Portada, Introducción, Objetivos, Selección de Herramientas).	CUMPLE 100%	Documento técnico estructurado con portada institucional, datos de la aprendiz en Popayán, Cauca, infografías vectoriales y maquetación en DesignCraft.

9. Conclusiones del Plan de Trabajo

1. **Gobernanza y Trazabilidad:** La implementación de Git y GitHub bajo el modelo GitFlow adaptado garantiza que cada módulo de la Pizzería Bella Italia se construya de forma ordenada, trazable y libre de sobreescrituras accidentales.

2. **Aseguramiento de Calidad Continua:** El pipeline automatizado con compuertas de calidad previene la introducción de bugs en producción y asegura que las 72 pruebas del proyecto se mantengan en verde antes de cada entrega.

3. **Viabilidad en Modalidad Virtual:** Estudiar 100% virtual desde Popayán, Cauca demuestra que el dominio de herramientas profesionales de control de versiones democratiza la participación en proyectos de software empresariales.

DOCUMENTO FIRMADO DIGITALMENTE POR:
Karen Yuliana Campo
Aprendiz ADSO • Ficha 228118 • Popayán, Cauca

CENTRO DE COMERCIO Y SERVICIOS
SENA REGIONAL CAUCA
Noviembre de 2026